hoRemote Data Scientist Assessment Test

Candidate Name: Jyothirmai Puram **Role:** Data Scientist
Company: Worthwhile
Time taken to solve: May 2–3, 2026
Contact: jyothirmaip18@gmail.com | +1 413-466-5227 | San Francisco, CA

Table of Contents

Part One - Candidate Questions and Technical Responses

Candidate's Name: Jyothirmai Puram

---

1. Are you currently employed?

Currently I am not employed.

---

2. How many hours can you devote weekly to the Company?

40 hours per week.

---

3. Why do you think you fit the role well? Please provide examples of your skills and experiences that align with the requirements of this role.

I think I am a strong fit because the work Worthwhile does - turning AI and data ideas into production with ready solutions which is exactly the kind of work I have been doing for the last four years. A few specific examples:

End-to-end Machine Learning (ML) delivery. At Hexaware Technologies, I led a customer churn prediction model using XGBoost (Extreme Gradient Boosting) that helped reduce client churn by 15%. This involved data cleaning, feature engineering, modeling, and rolling the model into a client-facing application - not just a notebook.

Cloud and Machine Learning Operations (MLOps). At MetLife, I build and deploy models in Amazon Web Services (AWS) SageMaker, with Redshift for data, Lambda and EC2 (Elastic Compute Cloud) for compute, and Continuous Integration / Continuous Deployment (CI/CD) pipelines for releases. This is the kind of "move into production, not pilots that go nowhere" delivery that Worthwhile talks about on its website.

Python, Structured Query Language (SQL), and modern ML stack. I work daily in Python with pandas, NumPy, scikit-learn, PyTorch, and TensorFlow, and I write SQL against Redshift and Relational Database Service (RDS) instances. I have cleaned and modeled on tens of thousands to billions of rows of data across insurance and retail use cases.

Big-data tooling. I am comfortable with Extract, Transform, Load (ETL) using SQL and Apache Spark, and I have working familiarity with the Hadoop ecosystem (Hadoop Distributed File System, Hive-style queries on top of distributed storage). I used Spark-based pipelines while working on the multi-billion-row Demand AI dataset at Hexaware, which is what makes me comfortable scaling beyond a single machine when a client's data calls for it.

Natural Language Processing (NLP) and Generative AI. I have built NLP modules for clustering, semantic similarity, and summarization of insurance documents, and I have used LangChain and Retrieval-Augmented Generation (RAG) pipelines for question-answering systems. This matches Worthwhile's focus on practical AI integration into client workflows.

Working with stakeholders. I have collaborated with data engineers, consultants, and product managers to integrate models into enterprise applications, and I have mentored interns. I am comfortable explaining models to non-technical leaders, which is important in a consulting setting.

Education. I completed my Master of Science in Computer Science at the University of Massachusetts, Amherst, where I did graduate research on Large Language Model (LLM) evaluation and multi-hop question answering.

---

4. Have you worked from home before?

Yes. Most of my recent work, including my role at MetLife and parts of my Hexaware engagements, has been remote or hybrid. I am set up with a dedicated workspace, reliable internet, and good habits around written updates, calendar discipline, and async communication.

---

5. What interests you about this role?

I like that Worthwhile focuses on solving real business problems instead of building pilots that never ship. The mix of AI strategy, data readiness, and engineering in one team matches how I prefer to work - go from a messy dataset to a model running in production, and stay involved long enough to see the business outcome.

---

6. What motivates you?

Two things motivate me the most: solving problems that change a real metric for the business, and learning from people who are stronger than me in some area. I get the most energy from projects where I can see the impact, like the churn model I worked on that moved retention by 15%.

---

7. Are you seeking employment in a company of a specific size, such as a small startup, medium-sized Company, or large corporation? Please explain your preference.

I am most drawn to small and mid-sized companies. I have worked at a large enterprise (MetLife) and a large IT services firm (Hexaware), and I have also worked in a research lab. I enjoy environments where I can own a problem end-to-end, talk to clients directly, and see the result of my work quickly. Worthwhile's size and consulting model fit that preference well.

---

8. Describe your experience working with a culturally diverse group of people.

I have worked with teammates and clients across India, the United States at Hexaware, and at UMass Amherst I collaborated with researchers from Graphite AI in the Information Extraction and Synthesis Laboratory (IESL). I am used to adjusting communication style, time zones, and meeting cadence to keep everyone aligned, and I default to written summaries so nothing depends on one shared accent or idiom.

9a. Given a dataset with customer transactions, identify three meaningful insights using Python or R. Explain your process and reasoning.

With a customer transactions dataset (columns like `customer_id`, `transaction_date`, `amount`, `product_category`, `channel`), I start by loading the data and confirming that `transaction_date` is parsed as a datetime type. Then I run summary statistics, check for missing values, and do a quick sanity check on `amount` - no negatives unless they are refunds. After that I move from descriptive to behavioral views: per-customer aggregates, time trends, and segmentation.

Insight 1 - Revenue is concentrated in a small share of customers.

```
import pandas as pd

df = pd.read_csv("transactions.csv", parse_dates=["transaction_date"])

cust_rev = df.groupby("customer_id")
["amount"].sum().sort_values(ascending=False)
top_20_pct = int(len(cust_rev) * 0.20)
share = cust_rev.head(top_20_pct).sum() / cust_rev.sum()
print(f"Top 20% of customers drive {share:.1%} of revenue")
```

This is a classic Pareto check. If the top 20% of customers drive 70–80% of revenue, retention and account management efforts should focus there first.

Insight 2 - Recency, Frequency, Monetary (RFM) segmentation surfaces at-risk customers.

```
snapshot = df["transaction_date"].max() + pd.Timedelta(days=1)
rfm = df.groupby("customer_id").agg(
    recency=("transaction_date", lambda x: (snapshot - x.max()).days),
    frequency=("transaction_date", "count"),
    monetary=("amount", "sum"),
)
rfm["R"] = pd.qcut(rfm["recency"], 4, labels=[4, 3, 2, 1]).astype(int)
rfm["F"] = pd.qcut(rfm["frequency"].rank(method="first"), 4, labels=[1, 2,
3, 4]).astype(int)
rfm["M"] = pd.qcut(rfm["monetary"], 4, labels=[1, 2, 3, 4]).astype(int)
rfm["score"] = rfm["R"] + rfm["F"] + rfm["M"]
```

Customers with high monetary and frequency scores but low recency are valuable customers who are quietly drifting away - a clear retention target.

Insight 3 - Seasonality and weekday patterns guide marketing timing.

```
df["month"] = df["transaction_date"].dt.month
df["dow"]   = df["transaction_date"].dt.day_name()

monthly = df.groupby("month")["amount"].sum()
dow     = df.groupby("dow")["amount"].mean()
```

If revenue spikes in Q4 or on weekends, that tells the business when to push promotions, when to staff support, and when to plan inventory. I would visualize these as a line chart for monthly totals and a bar chart for the day-of-week average ticket size.

9b. What steps would you take to detect and handle outliers in a dataset? Provide code snippets if possible.

My approach is to first decide whether the outlier is a data quality issue or a real but extreme observation, and then decide what to do about it.

Step 1 - Visual checks. Boxplots and histograms first, because they are quick and informative.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(x=df["amount"])
plt.title("Transaction amount - boxplot")
plt.show()
```

Step 2 - Statistical detection.

```
import numpy as np

q1 = df["amount"].quantile(0.25)
q3 = df["amount"].quantile(0.75)
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
iqr_outliers = df[(df["amount"] < lower) | (df["amount"] > upper)]

z = (df["amount"] - df["amount"].mean()) / df["amount"].std()
z_outliers = df[z.abs() > 3]
```

I use the Interquartile Range (IQR) rule when the variable is skewed, and the Z-score rule when it is roughly normal.

Step 3 - Business-rule checks. A negative `amount` on a non-refund row, or a transaction at 3 AM in a country where the store is closed, is almost always bad data. I encode these as explicit rules.

Step 4 - Handling. I choose based on context: drop obviously broken rows, cap (winsorize) extreme but plausible values at the 99th percentile, transform heavy-tailed variables with `np.log1p`, or keep and flag genuine high-value transactions. The key principle is to never silently drop outliers - document what was removed, why, and how many rows it affected.

```
df["amount_capped"] = df["amount"].clip(upper=df["amount"].quantile(0.99))
df["log_amount"]    = np.log1p(df["amount_capped"])
```

10a. You are given a dataset with labeled outcomes. Build a classification model of your choice and justify your selection. Provide code, evaluation metrics, and interpretation of the results.

For a typical tabular labeled dataset, I default to a Random Forest (RF) or Gradient Boosting model with a Logistic Regression baseline. Both handle mixed feature types, are robust to outliers, and give feature importance for free. I will use a Random Forest here because it is fast, has few assumptions, and is easy to explain to a business audience.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, RocCurveDisplay)
```

```
SEED = 42

df = pd.read_csv("labeled_dataset.csv")
y = df["target"]
X = df.drop(columns=["target"])

num_cols = X.select_dtypes(include=np.number).columns.tolist()
cat_cols = X.select_dtypes(exclude=np.number).columns.tolist()

pre = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=SEED
)

baseline = Pipeline([("pre", pre),
                     ("clf", LogisticRegression(max_iter=1000,
                                                class_weight="balanced",
                                                random_state=SEED))])
rf = Pipeline([("pre", pre),
               ("clf", RandomForestClassifier(n_estimators=400,
                                              min_samples_leaf=2,
                                              class_weight="balanced",
                                              n_jobs=-1,
                                              random_state=SEED))])

for name, model in [("Logistic Regression", baseline), ("Random Forest",
rf)]:
    model.fit(X_tr, y_tr)
    y_pred  = model.predict(X_te)
    y_proba = model.predict_proba(X_te)[:, 1]
    print(name)
    print(classification_report(y_te, y_pred, digits=3))
    print("ROC-AUC:", round(roc_auc_score(y_te, y_proba), 3))
    print("Confusion matrix:\n", confusion_matrix(y_te, y_pred))
```

How I read the metrics. Accuracy is only useful when classes are roughly balanced. Precision answers "of the customers I flagged, how many were truly positive?" - high precision means I am not wasting outreach. Recall answers "of the truly positive customers, how many did I catch?" - in churn or fraud, missing a positive is expensive, so I weight recall heavily. F1-score is the harmonic mean of the two, a useful single number when both matter. Receiver Operating Characteristic Area Under the Curve (ROC-AUC) is threshold-agnostic and tells me how well the model ranks positives above negatives. The confusion matrix is what I share with stakeholders because it makes the false-positive vs false-negative trade-off concrete.

Result interpretation. I compare the Random Forest against Logistic Regression on the same metrics. If the Random Forest beats the baseline meaningfully on ROC-AUC and recall, I keep it; if not, I prefer the simpler Logistic Regression because it is easier to explain and maintain. I also pull `feature_importances_` to confirm the top drivers make business sense before signing off.

10b. Explain how you would tune hyperparameters for a Random Forest model. What strategies or tools would you use?

A Random Forest is an ensemble of many decision trees. Each tree is trained on a random sample of rows and a random subset of features, and the forest averages their votes. The randomness reduces overfitting and makes it robust on tabular data.

The hyperparameters I tune most often are: `n_estimators` (number of trees - more is usually better up to a cost point), `max_depth` (limits tree depth; lower depth means more bias, less variance), `min_samples_split` and `min_samples_leaf` (minimum rows required to split or to be in a leaf - raising these regularizes the model), `max_features` (how many features each split considers - `sqrt`, `log2`, or a fraction), `class_weight` (important when classes are imbalanced), and `criterion` (gini or entropy - rarely changes much).

I usually start with a Randomized Search over a wide grid to find a good region, then a Grid Search over a narrow grid to fine-tune.

```
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold

param_dist = {
    "clf__n_estimators":     [200, 400, 600, 800],
    "clf__max_depth":        [None, 6, 10, 16, 24],
    "clf__min_samples_split":[2, 5, 10],
    "clf__min_samples_leaf": [1, 2, 4],
    "clf__max_features":     ["sqrt", "log2", 0.5],
    "clf__class_weight":     [None, "balanced"],
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
search = RandomizedSearchCV(
    rf, param_distributions=param_dist,
    n_iter=40, cv=cv, scoring="roc_auc",
    n_jobs=-1, random_state=SEED, verbose=1,
)
search.fit(X_tr, y_tr)
print(search.best_params_)
```

I use Stratified K-Fold (typically 5 folds) so that each fold preserves the class ratio. For larger search spaces I use Optuna with a Tree-structured Parzen Estimator (TPE) sampler - it is more sample-efficient than grid or random search. I also track runs with MLflow so I can compare experiments and reproduce the best run later.

Part Two - SQL, Technical Experience, Company Fit, and Churn Strategy

1. Write a SQL query to find the top 5 products by total sales from a table named sales with columns: product_id, sale_date, and amount.

```sql
SELECT
    product_id,
    SUM(amount) AS total_sales
FROM sales
GROUP BY product_id
ORDER BY total_sales DESC
LIMIT 5;
```

I aggregate `amount` per `product_id`, sort from highest to lowest total sales, and keep the top five. On SQL Server I would replace `LIMIT 5` with `TOP 5`. If the business cares about a specific window (for example, the last quarter), I would add a `WHERE sale_date >= DATEADD(quarter, -1, CURRENT_DATE)` filter, and I would wrap totals in `ROUND(..., 2)` for cleaner reporting.

---

2. Share a challenging technical issue you've encountered and how you resolved it, focusing on your analytical and critical thinking skills.

At Hexaware, I worked on the Demand AI forecasting pipeline that processes more than six billion global retail sales records. The problem was that the ingestion and disaggregation steps had become a bottleneck - runs were missing their service-level windows, and adding a new region to the pipeline was painful.

I profiled the pipeline end-to-end and broke runtime down by stage. The disaggregation step dominated, and inside it a small number of nested loops over product-region combinations were responsible for most of the time. I also checked memory pressure and noticed we were re-reading slices of data from storage instead of streaming them.

I rewrote the path in C++ with parallel processing, added an Application Programming Interface (API) for disaggregation so other teams could call it without rerunning the whole pipeline, and reorganized the data flow so each record was touched once. The result was a 30% reduction in processing time, a 15% increase in forecast accuracy across three new regions, and a 20% boost in user satisfaction from the downstream teams.

The lesson I took from it: profile before you optimize, and always check whether the bottleneck is algorithmic or just a bad memory access pattern.

---

3. How do you handle feedback and criticism from supervisors or colleagues?

I take it as useful information, not a personal attack. I listen first, restate what I heard, and then ask clarifying questions if needed. If I agree, I act on it quickly; if I disagree, I share my reasoning calmly and we decide together. Code reviews and design reviews have made me a better engineer, so I actively ask for them.

---

4. What is your greatest strength? How will it help you in this role?

My greatest strength is taking a messy problem and turning it into a working solution end-to-end - from data cleaning to a model that runs in production. In a consulting role at Worthwhile, that means I can move quickly from a client's vague question to a concrete, validated answer without getting stuck in any one stage.

---

5. We want to understand your long-term career goals and how this position aligns with them. Please share your aspirations and how you see this role contributing to your career path.

My long-term goal is to grow into a data scientist and eventually a technical lead who can bridge AI strategy and engineering for clients across industries. This role aligns well with that path because Worthwhile works directly with leadership teams, ships real software, and covers AI strategy through delivery - exactly the breadth I want to build.

---

6. What do you know about our Company?

From Worthwhile's website, I understand it is an AI and technology strategy consulting firm that works with founders and leaders at small to mid-sized companies. The work spans AI strategy and readiness, technology strategy and planning, custom software engineering, and modernization. Engagements often start with a three-week Pathfinder to align on what is worth pursuing before any large commitment, and the team emphasizes shipping production-grade work rather than pilots that go nowhere. If anything I have stated is out of date, I would be glad to be corrected during the interview.

---

7. How do you see yourself contributing to our Company's continued success and growth in this role?

I would contribute in three ways: shipping reliable models for clients (churn, fraud, forecasting, NLP) using the AWS and Python stack I already work in; helping clients understand what is realistic with AI so engagements stay grounded; and mentoring or supporting junior team members the way I did with interns at MetLife. The goal is repeat client work, not one-off pilots.

---

8. In your opinion, what are the biggest challenges facing our Company in the market?

I see three main challenges in the AI strategy consulting market right now. First, the gap between client expectations of Generative AI and what is actually production-ready - many clients want results faster than the data is ready for. Second, competition from in-house teams and large platform vendors that are bundling AI features into existing products. Third, talent pricing pressure, since strong AI engineers are expensive and clients are price-sensitive. The advantage of a firm like Worthwhile is the focus on what is worth building, which is exactly the answer to all three.

---

9. Discuss your familiarity with recent technological trends and how they can be leveraged within the industry.

The trends I work with most directly are Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), agentic frameworks like LangChain, vector databases such as Facebook AI Similarity Search (FAISS), and serverless ML on AWS SageMaker and Bedrock. On the data engineering side, I am familiar with Apache Spark for distributed ETL and the Hadoop ecosystem (Hadoop Distributed File System, MapReduce, Hive) for large historical datasets - both of which I have used on multi-billion-row retail

forecasting work. In a consulting setting, these are most useful when applied to narrow, valuable problems: summarizing long policy documents for compliance, building internal RAG search over a company's knowledge base, or using an LLM as a judge to evaluate model output. I am careful to recommend them only where they clearly beat a simpler approach.

---

10. Worthwhile wants to reduce customer churn. Outline your approach, including which data you'd need, which analyses/models you'd use, and how you'd communicate your findings to stakeholders.

The first thing I would do is agree on a precise definition of "churn" with leadership - is it a cancellation, a non-renewal, a drop in spend below a threshold, or a long inactivity gap? The model is only useful if it predicts the thing the business wants to act on.

Data I would need. Customer profile (tenure, plan type, geography, segment), usage and engagement (logins, feature usage, support tickets, response times), billing (monthly charges, payment method, late payments, refunds), lifecycle events (onboarding milestones, plan changes, complaints), and the historical churn label with date.

Exploratory Data Analysis (EDA). I check class balance, missingness, duplicates, and outliers; cross-tab churn against tenure, contract type, monthly charges, and support volume; and look for cohort effects.

Feature engineering. Tenure buckets (0–6 months, 6–12, 12–24, 24+), trends in usage over the last 30/60/90 days, support ticket aggregates, ratio features (monthly charges to total charges), and interaction flags like "month-to-month + high charges + low engagement."

Modeling options. Logistic Regression as a simple explainable baseline; Random Forest, Gradient Boosting, or XGBoost as a stronger model; and Cox Proportional Hazards or a survival random forest if leadership cares about *when* a customer will churn, not just whether.

Evaluation metrics. I lead with recall - we cannot afford to miss likely churners. I also watch precision so we do not blanket every customer with retention offers, and I report ROC-AUC and F1-score. I would also include lift at the top decile because it maps directly to a campaign budget.

Stakeholder communication. Three artifacts: a short executive summary with the headline numbers and the top three churn drivers; a notebook or dashboard for the analytics team showing the full evaluation; and a churn-risk dashboard for the customer success team that ranks customers by predicted risk and shows the top contributing features per customer.

Actionable recommendations. Target the top decile of risk with retention offers and personal outreach. Improve onboarding for early-tenure customers. Offer incentives or annual plans to month-to-month customers. Watch high-charge, low-engagement customers as a leading indicator. Set up monitoring so the model is retrained on a clear cadence and so we can A/B test retention offers against a control group.

Part Three - Remote Work, Teamwork, and Professional Fit

---

1. What tools or strategies are most effective for communication and collaboration in a remote work environment?

The tools I rely on are Slack or Microsoft Teams for chat, Zoom or Google Meet for calls, Jira or a shared project board for tasks, GitHub for code, and Confluence or Notion for documentation. The habits matter more than the tools though - clear written updates, decisions documented in writing, regular short check-ins instead of long meetings, and async-first communication so people in different time zones are not blocked.

---

2. If you were hiring for this role, which key personality traits and qualities would you look for?

I would look for clear written communication, intellectual honesty (the ability to say "I do not know" and then go find out), strong fundamentals in statistics and software engineering, comfort with ambiguity, and an instinct to tie analysis back to business value. Curiosity and reliability matter more to me than knowing every framework.

---

3. How do you approach problem-solving as a team member versus an individual contributor?

As an individual contributor, I prefer to think on paper first, draft a small plan, and then execute. As a team member, I do the same thinking but make it visible - share the plan early, ask for pushback, and split the work. I try to over-communicate when collaborating because it reduces rework later.

---

4. How do you prioritize and manage your workload in a fast-paced environment?

I rank work by impact and deadline, block focus time on the calendar for the hardest tasks, and keep a short list of "in progress" items so nothing slips. I review priorities at the start of each day and at the end of the week, and I flag risks early instead of waiting until something is actually late.

---

5. What specific skills or experiences do you hope to gain from this role to help you achieve your career goals?

I want more direct exposure to client-facing AI strategy work, more practice scoping engagements (especially the Pathfinder-style discovery), and more time delivering production AI across different industries. I also want to grow as a communicator who can translate model output into clear business decisions.

---

6. Imagine that you told a client you would be there at 10 am. It is now 10:30 am, and you won't finish your job until 11:30. You have a lunch meeting with another client at noon, followed by another job at 1:15 pm. How would you handle this situation?

I would call the current client right away, apologize, and give them a realistic finish time of 11:30. Then I would call the lunch client and either move the meeting to 12:30 or shorten it so the overrun does not cascade. I would also confirm the 1:15 pm job is still on track and message that client a brief heads-up if there is any risk of being late. After the day, I would document what caused the slip - wrong scope estimate, missing data - so the same mistake does not happen again. The principles are: communicate early, take ownership without over-explaining, and protect the next commitment.

7. In your experience, how frequently does the following problem occur: employees being afraid to express disagreement with their managers?

It happens fairly often, especially in larger or more hierarchical organizations. The teams I have worked best on were the ones where the manager actively asked for disagreement and rewarded honest pushback. As a team member, I try to make disagreement feel low-cost by framing it as a question or a trade-off rather than a confrontation.

---

8. Your experience working on projects involving a consortium of companies is valuable. Please briefly share your experience in this area.

At Hexaware, my IT consulting work regularly involved multiple organizations - internal product teams, the client's engineering and business teams, and sometimes a third-party vendor. I coordinated data access agreements, aligned model requirements across stakeholders, and integrated AI/ML models into client web applications. My role at MetLife also involves working across business, compliance, and engineering, which is structurally similar. I do not have a formal "consortium" engagement on my resume, so I want to be honest about that, but the multi-stakeholder coordination work is something I have done consistently.

---

9. How much would you request per hour?

I am happy to discuss based on scope, weekly hours, and whether this is hourly or a longer fixed engagement.

---

10. When would you be available to start if hired?

Immediate.

Technical Exercise

Customer Churn Prediction and Retention Analysis Using a Telecom Dataset

About This Project

Task: Analyzing a publicly available dataset, defining a business-relevant problem, building a model, and communicating findings like a working data scientist would. I chose the IBM Telco Customer Churn dataset from Kaggle - 7,043 customer rows and 21 columns - because churn prediction is directly relevant to Part Two Question 10 in this assessment, and it is one of the cleanest examples of how a well-built model can move a real business metric.

The goal I set for myself: predict which customers are likely to churn in the next month and identify the main factors driving that churn, so a customer success team can intervene before it happens.

I spent two days on this project. Below is what I did each day, what problems I ran into, and how I solved them.

Day 1: Understanding the Data (Exploratory Data Analysis and Preprocessing)

What I Did

I started by loading the dataset and just looking at it to understand what I was working with. The dataset has 7,043 rows and 21 columns. The target variable is `Churn` - Yes if the customer left within the last month, No otherwise.

The first thing I noticed was the class imbalance - 73.5% of customers do not churn and only 26.5% do. That is a roughly 3:1 ratio. It is not as extreme as some datasets I have worked with, but it still means that a naive model that always predicts "No churn" would get 73.5% accuracy while being completely useless for finding customers who are about to leave. So right away I knew I had to focus on recall and F1-score instead of accuracy, and I would use `class_weight="balanced"` to make the models pay extra attention to the minority class.

Figure 1: Churn distribution - 73.5% stay vs 26.5% churn. This imbalance made me focus on recall and F1-score rather than accuracy.

Churn distribution

Then I looked at potential data quality issues. The `TotalCharges` column had blank strings for a small number of customers with `tenure = 0` - these are brand-new customers who have not been billed yet. I coerced `TotalCharges` to numeric and filled those blanks with zero, which made sense since they genuinely had no charges yet.

I also noticed that several service-related columns like `OnlineSecurity`, `TechSupport`, and `StreamingTV` had three values: `Yes`, `No`, and `No internet service`. The third category is just a redundant way of saying `No` - if you do not have internet, you do not have these services. I collapsed those to `No` to simplify the encoding without losing any real information.

Next I looked at which features seem most useful for predicting churn:

Contract type turned out to be the strongest categorical signal. Month-to-month customers churn far more than one- or two-year contract customers. This makes intuitive sense - a long-term contract is both a commitment signal and a pricing incentive. When I saw this in the chart, my first business instinct was: one of the cheapest retention levers is offering a discount to move high-risk month-to-month customers onto an annual plan.

Figure 2: Churn by contract type. Month-to-month customers are churning at a dramatically higher rate than one- or two-year contract holders.

Churn by contract type

Tenure was the most telling numeric signal. Churn is heavily concentrated in the first few months and tapers off sharply after a year. Customers who survive the first twelve months are much more likely to stay. This told me that onboarding and the first 90 days matter enormously - a structured check-in program for new customers would likely pay off fast.

Figure 3: Churn by tenure (months). The churn spike in the early months is striking. Customers who make it past their first year rarely leave.

Churn by tenure

Monthly charges also showed a clear pattern. Customers on higher monthly plans churn more often. The churn density curve is shifted clearly to the right compared to non-churners. High-charge customers are paying more but may not feel they are getting proportional value - a pricing and value communication problem.

Figure 4: Monthly charges by churn. Churning customers are paying more per month on average, suggesting price sensitivity or perceived value mismatch.

Monthly charges by churn

I also ran a correlation check on the numeric features. As expected, `tenure` and `TotalCharges` are highly correlated - the longer you have been a customer, the more you have paid in total. This told me I needed to be careful not to use both features naively in a linear model without regularization, because they would give me near-duplicate information and inflate the importance of that axis.

Figure 5: Correlation heatmap. Tenure and TotalCharges are strongly correlated. MonthlyCharges is positively correlated with churn while tenure is negatively correlated.

Correlation heatmap

Preprocessing Steps I Built

Step	What I Did	Why
Fix TotalCharges	Coerced blank strings to numeric, filled new-customer zeros	Keeps those rows usable without fabricating a number
Collapse service categories	Replaced "No internet service" / "No phone service" with "No"	Removes redundant encoding
Encode target	Mapped Churn: Yes → 1, No → 0	Models need numeric input
Drop customerID	Removed the ID column	It carries no predictive signal
One-Hot Encoding	Applied to all remaining categorical columns via ColumnTransformer	Handles unknown categories at inference
Standard scaling	Applied to numeric features (tenure, MonthlyCharges, TotalCharges, SeniorCitizen)	Makes features comparable for the Logistic Regression baseline
Stratified 80/20 split	Split the data keeping the 26.5% churn ratio in both sets	Ensures fair evaluation on the test set

After preprocessing I had 5,634 training samples and 1,409 test samples, with the class ratio preserved in both.

Challenges I Faced on Day 1

1. Deciding what to do with the redundant service columns. There were several columns that said "No internet service" as a third category. At first I was not sure whether to keep them as three-level categoricals or collapse them. I decided to collapse to binary because the "No internet service" group is already captured by the `InternetService` column - keeping the redundancy would just make the One-Hot Encoding messier and less interpretable.

2. TotalCharges and tenure collinearity. When I saw the correlation heatmap I realized I had a real decision to make for the linear model - should I drop `TotalCharges` or keep both with regularization? I chose to keep both and rely on `StandardScaler` plus Logistic Regression's L2 penalty to manage the collinearity, rather than making a hard drop decision up front. Tree-based models do not care about this at all.

3. Figuring out what metric to optimize for. With 26.5% churn, accuracy is not the right metric. But I also had to decide between recall and F1 as my primary target. I landed on recall as the priority - missing a customer who is about to churn is a lost subscriber and lost revenue, which is more costly than accidentally offering a retention deal to someone who was going to stay. I kept an eye on precision so the campaign list stays manageable.

Day 2: Modeling, Evaluation, and Business Insights

What I Did

Because of the class imbalance I knew I could not just train a default model and call it done. I needed models that pay extra attention to the minority (churn) class. I decided to try two approaches and compare them:

Model	Why I Chose It	How It Handles Imbalance
Logistic Regression	Simple, fast, and transparent - a useful baseline that any stakeholder can understand	`class_weight="balanced"` makes it penalize missed churners more heavily
Gradient Boosting	Handles non-linear interactions well; typically one of the best out-of-the-box performers on tabular churn data	Default threshold of 0.5 can be tuned down to push recall up

I built both models inside scikit-learn `Pipeline` objects with a shared `ColumnTransformer` for preprocessing. This keeps the code clean and means both models get identical feature transformations - any difference in results is purely from the model, not the preprocessing.

I evaluated both on the same held-out test set of 1,409 customers (20% of the data, stratified). Here are the results:

Model	Accuracy	Precision (churn)	Recall (churn)	F1 (churn)	ROC-AUC
Logistic Regression (class-weighted)	0.739	0.505	**0.786**	0.615	0.842

Model	Accuracy	Precision (churn)	Recall (churn)	F1 (churn)	ROC-AUC
Gradient Boosting	**0.801**	**0.659**	0.516	0.579	**0.843**

Both models rank customers almost identically well - the Receiver Operating Characteristic Area Under the Curve (ROC-AUC) is 0.842 vs 0.843, essentially the same. But at the default 0.5 threshold they behave very differently. Gradient Boosting has higher accuracy and precision, meaning fewer false alarms, but it only catches 51.6% of the real churners. Logistic Regression catches 78.6% of real churners, which is what matters most for a retention campaign.

Figure 6: ROC curves for both models. The curves almost overlap at AUC ≈ 0.84, showing that the models have similar ranking ability. The threshold is what separates them.

ROC curves

I decided that for a churn-prevention deployment, the class-weighted Logistic Regression is the better choice at the default threshold, because recall is the priority. Alternatively, I would lower Gradient Boosting's threshold to somewhere around 0.35–0.40 to push its recall above 0.75 while keeping precision reasonable.

Figure 7: Confusion matrix for the Gradient Boosting model at the default 0.5 threshold. It correctly identifies 193 churners (true positives) but misses 181 (false negatives). Lowering the threshold would shift more of those 181 into the correctly-caught bucket.

Confusion matrix

Feature Importance - What Is Driving Churn?

I extracted feature importances from the Gradient Boosting model to understand what it was actually learning. The top drivers were:

1. **Contract type (Two-year vs Month-to-month)** - the single strongest signal. Customers on two-year contracts almost never churn.
2. **Tenure** - the longer a customer has been around, the less likely they are to leave.
3. **Monthly charges** - higher charges correlate with more churn.
4. **Internet service (Fiber optic)** - fiber customers churn more, likely due to a combination of higher price and service quality expectations.
5. **Total charges** - correlated with tenure, but the model still extracts additional signal from it.
6. **Add-on services (OnlineSecurity, TechSupport)** - customers with these services churn less. They are stickiness factors.

Figure 8: Top 15 feature importances from the Gradient Boosting model. Contract type, tenure, and monthly charges dominate. This is the chart I would walk business leadership through - it directly points to the retention levers.

Feature importance

This makes business sense, which is important. I always do a sanity check before I trust a model - if the top features are noise columns or things the business cannot act on, that is a red flag. Here, every top driver is something the business team can actually work with.

The highest-risk customer profile that emerges from this analysis: a new month-to-month customer (tenure under 12 months) on a fiber plan, paying high monthly charges, with no `OnlineSecurity` or `TechSupport` add-ons. These customers need proactive outreach in the first 90 days.

Business Insights and Recommendations

After working through everything, here are my concrete recommendations:

1. Focus retention budget on the top-decile risk customers. The model concentrates actual churners into a small, identifiable group. Rather than running a blanket campaign, contact the top 10–15% riskiest customers with a targeted offer. This is far more cost-efficient.

2. Invest in early-tenure onboarding. The tenure chart on Day 1 made this obvious. Most churn happens in the first year. A structured 30/60/90-day journey - check-in calls, guided setup, first-value milestones - should move the needle faster than any model.

3. Offer contract upgrade incentives to month-to-month customers flagged as high risk. Shifting even a fraction of these customers to a one-year plan dramatically reduces their churn probability. The incentive cost is almost certainly lower than the customer acquisition cost.

4. Bundle OnlineSecurity and TechSupport into mid-tier and high-tier plans. Customers with these add-ons churn less. If they are optional add-ons today, bundling them shifts customers into a stickier product tier.

5. Watch high-charge, low-engagement customers weekly. This group is expensive to lose. A simple weekly report flagging customers with high charges but declining usage is a low-effort early warning system.

6. Build a churn-risk dashboard for the customer success team. The model output should be visible to the people who can act on it - not buried in a notebook. A simple dashboard (Streamlit, Tableau, or Power BI) showing each customer's risk score and the top contributing factors would let the team prioritize outreach without needing to understand the model.

7. Set up A/B testing of retention offers. After deploying the model, randomly assign a held-out group of predicted churners to a control condition (no intervention). This is the only way to measure whether the retention program is actually saving customers, rather than crediting the model for people who were going to stay anyway.

Challenges I Faced on Day 2

1. Choosing between Logistic Regression and Gradient Boosting. Gradient Boosting had better accuracy and precision at the default threshold, but Logistic Regression had substantially better recall. These are genuinely different trade-offs and the right answer depends on the business context - specifically, the cost of a false positive (wasted retention offer) versus the cost of a false negative (lost customer). I documented both and explained the trade-off rather than picking one arbitrarily.

2. Feature importance interpretation. `TotalCharges` and `tenure` are highly correlated. In the feature importance chart, this means the model's importance scores for these two features can be unreliable individually - the importance gets split across both. I noted this in the interpretation and suggested that in a

follow-up, I would use SHAP (SHapley Additive exPlanations) values to get more reliable attribution, especially for correlated features.

3. Communicating the precision-recall trade-off to a non-technical audience. The confusion matrix numbers are intuitive - 193 caught, 181 missed - but explaining *why* we would want to lower the threshold (catching more churners at the cost of more false alarms) requires a concrete dollar framing. I would answer this by asking the client: "What does a false positive cost you in terms of retention offer value? What does a false negative cost you in lost monthly revenue for X months?" Once those numbers are on the table, the threshold decision becomes straightforward.

Key Decisions I Made (and Why)

What I Decided	Why
Focus on recall over accuracy	A 73.5% majority class means accuracy is a misleading metric. Recall directly measures what the business cares about - catching churners.
Keep class-weighted models, not oversampling	Simpler, does not create synthetic data, and works well on a 3:1 imbalance. SMOTE would be worth trying if the imbalance were 10:1 or worse.
Collapse "No internet service" to "No"	The information is already captured by the `InternetService` column. Keeping it as a third category would add noise, not signal.
Keep both Logistic Regression and Gradient Boosting	Both are valid depending on the threshold and cost trade-off. I wanted to give the business a choice with clear reasoning, not just deliver one number.
Report recall as primary metric	Missing a churner is losing a paying customer. The cost of a missed churner (lifetime value) almost always exceeds the cost of a false-positive retention offer.
Flag TotalCharges/tenure collinearity	I did not drop either feature, but I documented the issue so that anyone extending this work knows to use SHAP or partial dependence plots for reliable attribution.

Final Results Summary

Metric	Value
Dataset	IBM Telco Customer Churn (Kaggle)
Rows / Columns	7,043 / 21
Target variable	Churn (26.5% positive)
Train / test split	80/20 stratified
Best model for recall	Logistic Regression (class-weighted)
Recall on churn class	0.786 (catches 78.6% of true churners)

Metric	Value
Precision on churn class	0.505
F1 on churn class	0.615
ROC-AUC	0.842
Best model for precision	Gradient Boosting
ROC-AUC (GB)	0.843
Top churn driver	Contract type (month-to-month)
Second top driver	Tenure (low = high risk)

Next Steps

1. **Hyperparameter tuning** with Randomized Search over Gradient Boosting and Random Forest, scoring on recall or F1, to see if a better-tuned tree model can match the Logistic Regression recall while improving precision.
2. **Threshold sweep** on Gradient Boosting to find the operating point that maximizes F1 or hits a target recall of ≥ 0.75, and compare that operating point to the Logistic Regression baseline.
3. **SHAP analysis** for more reliable feature attribution, especially for the correlated `tenure` / `TotalCharges` pair.
4. **Additional feature engineering** - cohort features, tenure buckets, recent-vs-historical usage trends, support ticket aggregates.
5. **Model monitoring and retraining** - track drift in feature distributions and in predicted churn rate, with a clear retraining cadence (monthly or quarterly).
6. **Stakeholder dashboard** - a lightweight Streamlit or Power BI dashboard that customer success can use daily, showing each customer's risk score and the top two contributing factors.
7. **A/B testing** of retention strategies to measure causal impact, not just predicted risk.

Appendix

A. Full Python Code

```
# Customer Churn Prediction – Telco Customer Churn (Public Dataset)
# Author: Jyothirmai Puram
# Dataset: https://www.kaggle.com/datasets/blastchar/telco-customer-churn

# --- Imports ---
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import (train_test_split, StratifiedKFold,
                                     RandomizedSearchCV)
```

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier,
RandomForestClassifier
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, RocCurveDisplay,
ConfusionMatrixDisplay)

SEED = 42
np.random.seed(SEED)
sns.set_theme(style="whitegrid")

# --- Load data ---
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
print("Shape:", df.shape)

# --- Clean ---
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"],
errors="coerce").fillna(0)

service_cols = ["OnlineSecurity", "OnlineBackup", "DeviceProtection",
                "TechSupport", "StreamingTV", "StreamingMovies",
"MultipleLines"]
for c in service_cols:
    df[c] = df[c].replace({"No internet service": "No", "No phone
service": "No"})

df["Churn"] = (df["Churn"] == "Yes").astype(int)
df = df.drop(columns=["customerID"])

# --- Split ---
y = df["Churn"]
X = df.drop(columns=["Churn"])
num_cols = ["tenure", "MonthlyCharges", "TotalCharges", "SeniorCitizen"]
cat_cols  = [c for c in X.columns if c not in num_cols]

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=SEED
)

# --- Visualizations (Day 1) ---
sns.countplot(x=y.map({0: "No", 1: "Yes"}))
plt.title("Figure 1: Churn distribution")
plt.show()

sns.countplot(data=df, x="Contract", hue=df["Churn"].map({0: "No", 1:
"Yes"}))
plt.title("Figure 2: Churn by contract type")
plt.show()

sns.histplot(data=df, x="tenure",
             hue=df["Churn"].map({0: "No", 1: "Yes"}),
```

```
              multiple="stack", bins=30)
plt.title("Figure 3: Churn by tenure (months)")
plt.show()

sns.kdeplot(data=df, x="MonthlyCharges",
            hue=df["Churn"].map({0: "No", 1: "Yes"}),
            common_norm=False, fill=True)
plt.title("Figure 4: Monthly charges by churn")
plt.show()

plt.figure(figsize=(6, 4))
sns.heatmap(df[num_cols + ["Churn"]].corr(), annot=True, cmap="Blues")
plt.title("Figure 5: Correlation heatmap (numeric features)")
plt.show()

# --- Preprocessing pipeline ---
preprocess = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])

# --- Models (Day 2) ---
logreg = Pipeline([
    ("pre", preprocess),
    ("clf", LogisticRegression(max_iter=1000, class_weight="balanced",
                               random_state=SEED)),
])

gb = Pipeline([
    ("pre", preprocess),
    ("clf", GradientBoostingClassifier(random_state=SEED)),
])

# --- Fit and evaluate ---
results = {}
for name, model in [("Logistic Regression", logreg), ("Gradient Boosting",
gb)]:
    model.fit(X_tr, y_tr)
    y_pred  = model.predict(X_te)
    y_proba = model.predict_proba(X_te)[:, 1]
    auc = roc_auc_score(y_te, y_proba)
    print(f"=== {name} ===")
    print(classification_report(y_te, y_pred, digits=3))
    print(f"ROC-AUC: {auc:.3f}\n")
    results[name] = {"model": model, "auc": auc,
                     "y_pred": y_pred, "y_proba": y_proba}

# --- Figure 6: ROC curves ---
fig, ax = plt.subplots()
for name, r in results.items():
    RocCurveDisplay.from_predictions(y_te, r["y_proba"], name=name, ax=ax)
plt.title("Figure 6: ROC curves - Logistic Regression vs Gradient
Boosting")
plt.show()
```

```
# --- Figure 7: Confusion matrix (Gradient Boosting) ---
ConfusionMatrixDisplay.from_predictions(
    y_te, results["Gradient Boosting"]["y_pred"],
    display_labels=["No churn", "Churn"], cmap="Blues"
)
plt.title("Figure 7: Confusion matrix - Gradient Boosting")
plt.show()

# --- Figure 8: Feature importance ---
gb_model = results["Gradient Boosting"]["model"]
ohe = gb_model.named_steps["pre"].named_transformers_["cat"]
feature_names = num_cols + list(ohe.get_feature_names_out(cat_cols))
importances   = gb_model.named_steps["clf"].feature_importances_
fi = pd.Series(importances,
index=feature_names).sort_values(ascending=False).head(15)

plt.figure(figsize=(7, 5))
fi[::-1].plot(kind="barh", color="#1f77b4")
plt.title("Figure 8: Top 15 features - Gradient Boosting")
plt.xlabel("Importance")
plt.tight_layout()
plt.show()

# --- Optional: Random Forest hyperparameter tuning sketch ---
rf = Pipeline([
    ("pre", preprocess),
    ("clf", RandomForestClassifier(class_weight="balanced",
                                   random_state=SEED, n_jobs=-1)),
])
param_dist = {
    "clf__n_estimators":     [200, 400, 600, 800],
    "clf__max_depth":        [None, 6, 10, 16, 24],
    "clf__min_samples_split":[2, 5, 10],
    "clf__min_samples_leaf": [1, 2, 4],
    "clf__max_features":     ["sqrt", "log2", 0.5],
}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
search = RandomizedSearchCV(rf, param_distributions=param_dist,
                            n_iter=20, cv=cv, scoring="roc_auc",
                            n_jobs=-1, random_state=SEED, verbose=1)
# Uncomment to run:
# search.fit(X_tr, y_tr)
# print("Best params:", search.best_params_)
# print("Best CV ROC-AUC:", round(search.best_score_, 3))
```

B. SQL Query

```
-- Top 5 products by total sales
-- Works on PostgreSQL, MySQL, SQLite, Snowflake, Redshift, BigQuery.
-- Replace LIMIT 5 with TOP 5 for SQL Server.
```

```sql
SELECT
    product_id,
    SUM(amount) AS total_sales
FROM sales
GROUP BY product_id
ORDER BY total_sales DESC
LIMIT 5;
```

C. Slide Outline (3–5 slides)

- **Slide 1 - Problem and Dataset.** Telco Customer Churn (7,043 rows, 21 columns). Goal: predict next-month churn and identify drivers. Business value: retention, reduced acquisition cost, targeted outreach.
- **Slide 2 - Exploratory Data Analysis Findings.** 26.5% churn rate. Strongest signals: month-to-month contracts, low tenure, high monthly charges, no security/tech-support add-ons. Key chart: churn by contract type (Figure 2) and churn by tenure (Figure 3).
- **Slide 3 - Model Approach and Metrics.** Logistic Regression baseline vs Gradient Boosting. Stratified 80/20 split, class-weighted models. Logistic Regression: Recall 0.786, ROC-AUC 0.842. Gradient Boosting: Recall 0.516, ROC-AUC 0.843. Recall prioritized - missing a churner is losing a paying customer.
- **Slide 4 - Key Insights.** Top drivers: contract type, tenure, monthly charges, fiber internet, add-on services. High-risk profile: new month-to-month fiber customer paying high charges with no add-ons.
- **Slide 5 - Recommendations and Next Steps.** Target top 10% risk with proactive outreach. Onboarding investment in the first 90 days. Contract-upgrade incentives. Churn-risk dashboard for customer success. A/B test retention offers to measure real impact.

D. GitHub Note

I can provide a clean Jupyter notebook of this churn analysis (`churn_analysis.ipynb`), the SQL query (`top_products.sql`), and the figure-generation script (`generate_figures.py`) as a single zip for the hiring panel.

References

1. IBM Sample Data / Kaggle - Telco Customer Churn dataset. https://www.kaggle.com/datasets/blastchar/telco-customer-churn
2. Scikit-learn documentation - https://scikit-learn.org/stable/
3. Worthwhile company overview - https://www.worthwhile.com/
4. Handling class imbalance in classification - https://scikit-learn.org/stable/modules/ensemble.html
5. Random Forest vs Gradient Boosting for tabular data - https://scikit-learn.org/stable/modules/ensemble.html

---

Jyothirmai Puram - Worthwhile Remote Data Scientist Assessment